

- JS7-继承
 - 1 原型
 - 1.1 原型对象 (prototype)
 - 1.2 内部原型 (___proto___)
 - 1.3 构造函数 (constructor)
 - 2 原型链
 - 2.1 链式结构
 - 2.2 重要方法
 - 1. hasOwnProperty
 - 2. in 操作符
 - 3. instanceof
 - 4. Object.create(proto)
 - 3 几种常见的继承模式（基于原型）
 - 3.1 原型链继承
 - 3.2 构造函数继承（经典借用）
 - 3.3 组合继承（最常用）
 - 3.4 寄生组合式继承（最优）
 - 3.5 ES6 class / extends
 - 4 常见考点与陷阱
 - 1. Function.__proto__ 与 Function.prototype
 - 2. Object.prototype.__proto__ 是 null
 - 3. instanceof 的坑
 - 4. 普通对象与函数对象的原型差异
 - 5. new 操作符的手写实现

JS7-继承

1 原型

1.1 原型对象 (prototype)

- 每个函数都有一个 **prototype** 属性（箭头函数等除外），它指向一个对象。
- 这个对象称为“原型对象”，主要用途是 让所有实例共享它的属性和方法。

```
function Person(name) {
  this.name = name;
}
Person.prototype.sayHi = function() { console.log('Hi' + ' ' + this.name); };

let p = new Person('petty').sayHi();    // Hi petty
```

1.2 内部原型 (___proto___)

- 每个 **对象**（包括函数对象）都有一个 `___proto___` 属性（非标准，但所有主流引擎都支持），指向其构造函数的 `prototype`。
- 标准写法：`Object.getPrototypeOf(obj)`

```
function Person(name) {
  this.name = name;
}
const p = new Person();
console.log(p.__proto__ === Person.prototype); // true
```

1.3 构造函数 (constructor)

- 原型对象默认有一个 `constructor` 属性，指向构造函数本身。
- 实例可以通过原型链找到 `constructor`。

```
function Person(name) {
  this.name = name;
}
const p = new Person();
console.log(Person.prototype.constructor === Person); // true
console.log(p.constructor === Person); // true
```

2 原型链

2.1 链式结构

- 当访问一个对象的属性/方法时，JS 引擎会按以下顺序查找：

1. 对象自身 (`hasOwnProperty`)
 2. 对象的 `__proto__` (即构造函数的 `prototype`)
 3. 继续向上, `__proto__.__proto__`
 4. 直到某个 `__proto__` 为 `null`, 结束查找。
- 这一层层链接形成的链条就是 **原型链**。

```
function Animal() {}
Animal.prototype.eat = function() {};

function Dog() {}
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.bark = function() {};

const d = new Dog();

// 原型链: d -> Dog.prototype -> Animal.prototype -> Object.prototype -> null
console.log(d.__proto__ === Dog.prototype);
console.log(d.__proto__.__proto__ === Animal.prototype);
console.log(d.__proto__.__proto__.__proto__ === Object.prototype);
console.log(d.__proto__.__proto__.__proto__.__proto__ === null);
```

属性遮蔽

- 实例自身的属性会“遮蔽”原型链上同名的属性。

```
function Foo() {}
Foo.prototype.name = 'prototype';

const f = new Foo();
f.name = 'instance';
console.log(f.name); // 'instance' (自身属性)
delete f.name;
console.log(f.name); // 'prototype' (原型上的)
```

2.2 重要方法

1. `hasOwnProperty`

- 判断属性是否是**对象自身**拥有的 (不来自原型链)。

```
f.hasOwnProperty('name'); // false (如果name来自原型)
```

2. `in` 操作符

- 判断属性是否 **可被对象访问到**（包括原型链）。

```
'name' in f;    // true（即使来自原型）
```

3. `instanceof`

- 判断一个对象的**原型链**上是否存在某个构造函数的 `prototype`。

```
d instanceof Dog;    // true
d instanceof Animal;  // true
d instanceof Object;  // true
```

4. `Object.create(proto)`

- 创建一个新对象，其 `__proto__` 指向传入的 `proto`。常用于实现继承

3 几种常见的继承模式（基于原型）

3.1 原型链继承

```
Child.prototype = new Parent();
```

- 缺点**：所有子实例会共享父实例的引用属性（如数组）。

3.2 构造函数继承（经典借用）

```
function Child() {
  Parent.call(this);
}
```

- 缺点**：无法继承父类原型上的方法。

3.3 组合继承（最常用）

```
function Child() {  
  Parent.call(this); // 继承属性  
}  
Child.prototype = new Parent(); // 继承方法  
Child.prototype.constructor = Child;
```

- 缺点：父构造函数被调用了两次。

3.4 寄生组合式继承（最优）

```
function inherit(child, parent) {  
  const prototype = Object.create(parent.prototype);  
  prototype.constructor = child;  
  child.prototype = prototype;  
}  
function Child() { Parent.call(this); }  
inherit(Child, Parent);
```

- 这是业界推荐的 **最佳实践**，也是牛客手写题常考内容。

3.5 ES6 `class / extends`

- 本质仍是原型链的语法糖。

```
class Parent { }  
class Child extends Parent { }
```

4 常见考点与陷阱

1. `Function.__proto__` 与 `Function.prototype`

```
console.log(Function.__proto__ === Function.prototype); // true
console.log(Object.__proto__ === Function.prototype);    // true
console.log(Function.prototype.__proto__ === Object.prototype); // true
```

- 所有函数（包括 `Object`、`Function` 本身）的 `__proto__` 都指向 `Function.prototype`。

2. `Object.prototype.__proto__` 是 `null`

```
console.log(Object.prototype.__proto__); // null
```

- 原型链的终点。

3. `instanceof` 的坑

```
function Foo() {}
const f = new Foo();
console.log(f instanceof Foo);    // true
console.log(f instanceof Object); // true
// 如果修改原型链：
Foo.prototype = {};
console.log(f instanceof Foo);    // false（因为f.__proto__仍指向旧原型）
```

4. 普通对象与函数对象的原型差异

```
const obj = {};    // __proto__ → Object.prototype
function func() {} // __proto__ → Function.prototype
```

5. `new` 操作符的手写实现

```
function myNew(Constructor, ...args) {
  const obj = Object.create(Constructor.prototype);
  const ret = Constructor.apply(obj, args);
```

```
return ret instanceof Object ? ret : obj;  
}
```